

✓ Data Cleaning

```
import numpy as np
import random

N_number = 14758736
np.random.seed(N_number)
random.seed(N_number)
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import pandas as pd

num_file_path = '/content/drive/MyDrive/PScapstone/rmpCapstoneNum.csv'
num_data = pd.read_csv(num_file_path, header=None)

qual_file_path = '/content/drive/MyDrive/PScapstone/rmpCapstoneQual.csv'
qual_data = pd.read_csv(qual_file_path, header = None)
```

```
#add column name for the file with number (ratings)
num_data.columns = [
    "Average Rating", "Average Difficulty", "Number of Ratings",
    "Received a Pepper", "Proportion Take Again", "Number of Online Reviews",
    "Male Gender", "Female Gender"
]
#add column name for the file with University info
qual_data.columns = ["Major/Field", "University", "US State"]
#check file and general descriptive statistics
print(num_data.head())
print(qual_data.head())
print(num_data.describe())
print(qual_data.describe())
# merge two data sheet into one, easier for manipulation
merged_data = pd.concat([num_data, qual_data], axis=1)
print(merged_data.describe(include="all"))
```

	Average Rating	Average Difficulty	Number of Ratings	Received a
0	5.0	1.5	2.0	
1	NaN	NaN	NaN	
2	3.2	3.0	4.0	

3	3.6	3.5	10.0
4	1.0	5.0	1.0

	Proportion Take Again	Number of Online Ratings	Male Gender	Female Gender
0	NaN	0.0	0	
1	NaN	NaN	0	
2	NaN	0.0	1	
3	NaN	0.0	0	
4	NaN	0.0	0	

	Major/Field	University	US State
0	Criminal Justice	George Mason University	VA
1	NaN	NaN	NaN
2	English	Alabama State University	AL
3	English	University of Kentucky	KY
4	English	Keiser University	FL

	Average Rating	Average Difficulty	Number of Ratings	\
count	70004.000000	70004.000000	70004.000000	
mean	3.808014	2.864623	5.374721	
std	1.126895	0.991064	8.136628	
min	1.000000	1.000000	1.000000	
25%	3.000000	2.000000	1.000000	
50%	4.000000	3.000000	3.000000	
75%	4.800000	3.500000	6.000000	
max	5.000000	5.000000	393.000000	

	Received a Pepper	Proportion Take Again	Number of Online Rat
count	70004.000000	12160.000000	70004.00
mean	0.279927	76.417105	0.31
std	0.448966	25.011441	1.05
min	0.000000	4.000000	0.00
25%	0.000000	60.000000	0.00
50%	0.000000	83.000000	0.00
75%	1.000000	100.000000	0.00
max	1.000000	100.000000	19.00

	Male Gender	Female Gender
count	89893.000000	89893.000000
mean	0.326789	0.301903
std	0.469042	0.459086
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	1.000000	1.000000
max	1.000000	1.000000

	Major/Field	University	US State
count	70004	70004	70004
unique	1272	3107	89
top	English	Arizona State University	CA
freq	6398	484	8514

	Average Rating	Average Difficulty	Number of Ratings	\
count	70004.000000	70004.000000	70004.000000	
unique	NaN	NaN	NaN	
top	NaN	NaN	NaN	
freq	NaN	NaN	NaN	

freq	freq	freq	freq
mean	2 808011	2 861622	5 271721

```
print(qual_data.describe())
```

	Major/Field	University	US	State
count	70004	70004	70004	
unique	1272	3107	89	
top	English	Arizona State University	CA	
freq	6398	484	8514	

```
#noticed that the following columns all have nan value on the same row
check1=merged_data[merged_data[["Average Rating", "Average Difficulty",
    "Received a Pepper", "Number of Online Ratings", "Major/Field", "University", "US State"]
    .isnull().any()]]

print(check1)
```

	Average Rating	Average Difficulty	Number of Ratings \
1	NaN	NaN	NaN
13	NaN	NaN	NaN
14	NaN	NaN	NaN
15	NaN	NaN	NaN
16	NaN	NaN	NaN
...	...	...	...
89879	NaN	NaN	NaN
89881	NaN	NaN	NaN
89886	NaN	NaN	NaN
89887	NaN	NaN	NaN
89892	NaN	NaN	NaN

	Received a Pepper	Proportion Take Again	Number of Online Ratings
1	NaN	NaN	NaN
13	NaN	NaN	NaN
14	NaN	NaN	NaN
15	NaN	NaN	NaN
16	NaN	NaN	NaN
...	...	...	...
89879	NaN	NaN	NaN
89881	NaN	NaN	NaN
89886	NaN	NaN	NaN
89887	NaN	NaN	NaN
89892	NaN	NaN	NaN

	Male Gender	Female Gender	Major/Field	University	US State
1	0	0	NaN	NaN	NaN
13	0	0	NaN	NaN	NaN
14	0	0	NaN	NaN	NaN
15	0	0	NaN	NaN	NaN
16	0	0	NaN	NaN	NaN
...	...	...	...	...	...
89879	0	0	NaN	NaN	NaN
89881	0	0	NaN	NaN	NaN
89886	0	0	NaN	NaN	NaN
89887	0	0	NaN	NaN	NaN
89892	0	0	NaN	NaN	NaN

```
[19889 rows x 11 columns]
```

```
#Then we drop those rows since there are just nan value and meaningless
merged_data_cleaned = merged_data.dropna(subset=["Average Rating", "Average Difficulty",
"Received a Pepper", "Number of Online Ratings", "Major/Field", "University", "US State"])
merged_data_cleaned.isnull().sum()
```

	0
Average Rating	0
Average Difficulty	0
Number of Ratings	0
Received a Pepper	0
Proportion Take Again	57844
Number of Online Ratings	0
Male Gender	0
Female Gender	0
Major/Field	0
University	0
US State	0

dtype: int64

```
# Handle missing values in "Proportion Take Again" by filling them with 0
# This avoids dropping rows unnecessarily as this column might be useful later
merged_data_cleaned["Proportion Take Again"] = merged_data_cleaned["Proportion Take Again"].fillna(0)
#check the cleaned data again.
print(merged_data_cleaned.describe())
```

	Average Rating	Average Difficulty	Number of Ratings \
count	70004.000000	70004.000000	70004.000000
mean	3.808014	2.864623	5.374721
std	1.126895	0.991064	8.136628
min	1.000000	1.000000	1.000000
25%	3.000000	2.000000	1.000000
50%	4.000000	3.000000	3.000000
75%	4.800000	3.500000	6.000000
max	5.000000	5.000000	393.000000

	Received a Pepper	Proportion Take Again	Number of Online Ratings
count	70004.000000	70004.000000	70004.000000
mean	0.279927	81.856522	0.315000
std	0.448966	10.718077	1.054000
min	0.000000	4.000000	0.000000
25%	0.000000	83.000000	0.000000
50%	0.000000	83.000000	0.000000
75%	1.000000	83.000000	0.000000
max	1.000000	100.000000	19.000000

	Male Gender	Female Gender
count	70004.000000	70004.000000
mean	0.419633	0.387678
std	0.493502	0.487224
min	0.000000	0.000000
25%	0.000000	0.000000
50%	0.000000	0.000000
75%	1.000000	1.000000
max	1.000000	1.000000

```
/tmp/ipython-input-1006885744.py:3: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: <https://pandas.pydata.org/pandas-docs/stable/10min.html#copy-on-write>

```
merged_data_cleaned["Proportion Take Again"] = merged_data_cleaned["Proportion Take Again"].fillna(0)
```

```
##### !!!!!!!!!!!!!!!
# important step! The project prompt, consideration part, professor as
# the fact that "it is more meaningful if the average rating is based
# For my consideration, accepting all data will result extreme rating
# and this might caused by students' bias. And for the method "Weight
# Using the number of ratings as weights introduces the information al
# into the "rating" variable. This can lead to inaccurate analysis bec
# "rating" may be confounded by the additional information from the nu

# Below was my attempt to test what the data will be like using the me
# I set the weight for each average rating as its number of rating/to
# but this introduce confounding effect as subsequent evaluation of r
# additional information from the number of ratings

Weighted_merged_data_cleaned = merged_data_cleaned.copy()

Weighted_merged_data_cleaned["Weighted Average Rating"] = Weighted_me

print(Weighted_merged_data_cleaned["Weighted Average Rating"])

### Conclusion, thus I used the set a threshold, and set the threshold
merged_data_cleaned = merged_data_cleaned[merged_data_cleaned['Number
```

```
0      0.000027
2      0.000034
3      0.000096
4      0.000003
5      0.000205
...
89885  0.000041
89888  0.000013
89889  0.000024
89890  0.000080
89891  0.000048
Name: Weighted Average Rating, Length: 70004, dtype: float64
```

✓ Analysis 1

Activists have asserted that there is a strong gender bias in student evaluations of professors, with male professors enjoying a boost in rating from this bias. While this has been celebrated by ideologues, skeptics have pointed out that this research is of technically poor quality, either due to a low sample size – as small as $n = 1$ (Mitchell & Martin, 2018), failure to control for confounders such as teaching experience (Centra & Gaubatz, 2000) or obvious p-hacking (MacNeill et al., 2015). We would like you to answer the question whether there is evidence of a pro-male gender bias in this dataset. Hint: A significance test is probably required.

```
print()
print("Analysis 1")
print()
from scipy.stats import shapiro
# Split data into male and female groups
male_ratings = merged_data_cleaned[merged_data_cleaned["Male Gender"]]
female_ratings = merged_data_cleaned[merged_data_cleaned["Female Gender"]]
print("median of male Average rating:", male_ratings.median())
print("median of female Average rating:", female_ratings.median())
```

Analysis 1

```
median of male Average rating: 4.2
median of female Average rating: 4.1
```

```
# Test normality of the distributions for male and female ratings
shapiro_test_male = shapiro(male_ratings)
shapiro_test_female = shapiro(female_ratings)
```

```
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:
    res = hypotest_fun_out(*samples, **kwargs)
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:
    res = hypotest_fun_out(*samples, **kwargs)
```

```
#or test whether both data is normally distributed by graph
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(12, 6))
```

```
sns.kdeplot(male_ratings, label="Male Ratings", shade=True, color="blue")
sns.kdeplot(female_ratings, label="Female Ratings", shade=True, color="red")
plt.title("Distribution of Average Rating")
plt.legend()
plt.show()
```

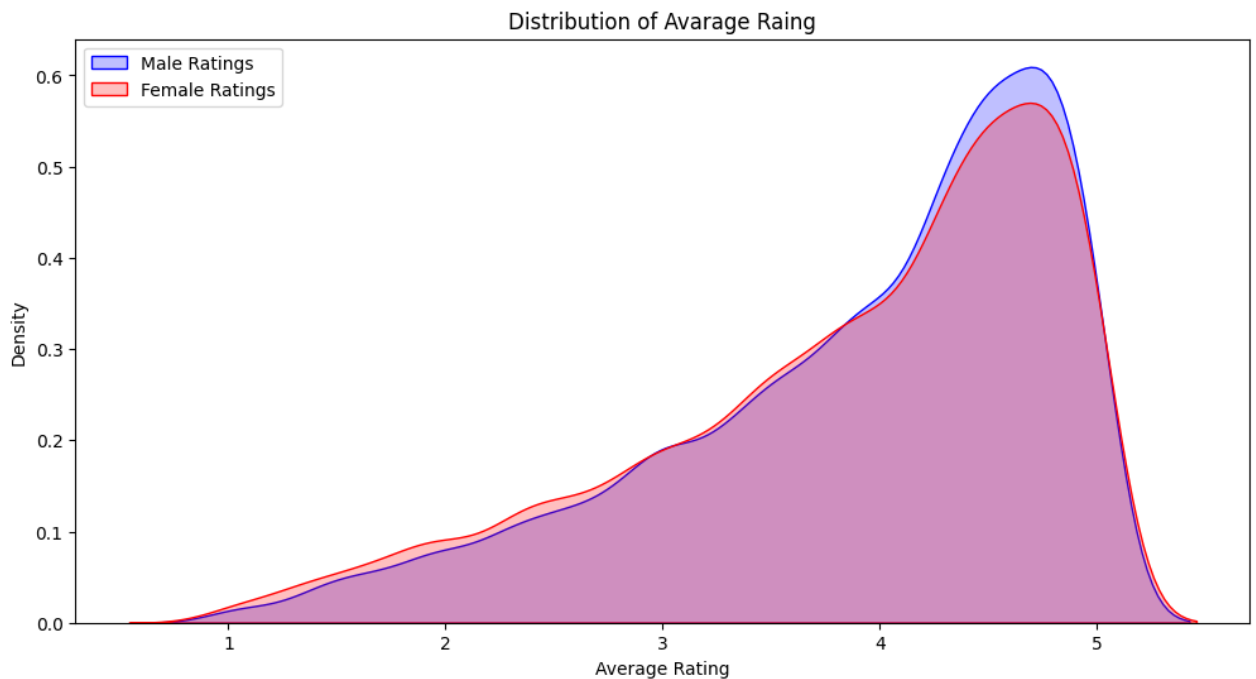
/tmp/ipython-input-1343082240.py:5: FutureWarning:

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(male_ratings, label="Male Ratings", shade=True, color="blue")
/tmp/ipython-input-1343082240.py:6: FutureWarning:
```

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(female_ratings, label="Female Ratings", shade=True, color="red")
```



```
# Print p-values of normality tests
#both group are not normally distributed, thus using nonparametric test
print(shapiro_test_male.pvalue, shapiro_test_female.pvalue)
#I planned to use mannwhitneyu test as:
from scipy.stats import mannwhitneyu
u_stats, u_p_value = mannwhitneyu(male_ratings, female_ratings)
"Mann-Whitney U-test", u_stats, u_p_value
print("u_stats ",u_stats)
print("u_p_value ",u_p_value)
#conclusion: Since the p value is extremely small, we drop the null hypothesis
#the male professor's average rating is higher than female.
```

```
9.940160349796176e-63 1.482117205346278e-58
u_stats 50901762.5
u_p_value 0.0008387610375322853
```

```
# Not done yet, since other factors like "Average Difficulty", "Number of Ratings"
# might also includes the average number of rating, we need to examine
import numpy as np
import statsmodels.api as sm
from sklearn.preprocessing import MinMaxScaler
import seaborn as sns
import matplotlib.pyplot as plt
X = merged_data_cleaned[["Male Gender", "Female Gender", "Average Difficulty", "Number of Ratings", "Received a Pepper", "Number of Online Ratings", "Proportion Take Again"]]
y = merged_data_cleaned["Average Rating"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print("Below is the model summary of the un-normalized model (I will normalize it later)")
print(model.summary())
```

Below is the model summary of the un-normalized model (I will normalize it later)

OLS Regression Results

```
=====
Dep. Variable:          Average Rating      R-squared:
Model:                  OLS                Adj. R-squared:
Method:                 Least Squares       F-statistic:
Date:                  Mon, 27 Oct 2025     Prob (F-statistic):
Time:                  19:24:49            Log-Likelihood:
No. Observations:      25368              AIC:
Df Residuals:          25360              BIC:
Df Model:               7
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t
const	3.8448	0.029	131.664	0.000
Male Gender	0.1061	0.009	11.355	0.000
Female Gender	0.0585	0.010	6.078	0.000
Average Difficulty	-0.5360	0.005	-101.882	0.000
Number of Ratings	0.0036	0.000	10.274	0.000
Received a Pepper	0.4968	0.009	58.174	0.000
Number of Online Ratings	-0.0072	0.003	-2.831	0.005
Proportion Take Again	0.0157	0.000	64.547	0.000

```
=====
Omnibus:                1971.619      Durbin-Watson:
Prob(Omnibus):           0.000        Jarque-Bera (JB):
Skew:                   -0.658        Prob(JB):
Kurtosis:               3.920         Cond. No.
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is

```
# Create a copy of the cleaned dataset for normalization
normalized_data = merged_data_cleaned.copy()
columns_to_normalize = ["Male Gender", "Female Gender", "Average Difficulty", "Number of Ratings", "Received a Pepper", "Number of Online Ratings", "Proportion Take Again"]
```

```

scaler = MinMaxScaler()
# Apply normalization to the selected columns
normalized_data[columns_to_normalize] = scaler.fit_transform(normalized_data[columns_to_normalize])
print("normalized data:", normalized_data[columns_to_normalize].head())

# normalize data to ensure the factors are in the same scale, so we can
X = normalized_data[columns_to_normalize]
y = normalized_data["Average Rating"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print("below is the summary of this normalized multiple regression model")
print(model.summary())

```

```

normalized data:      Male Gender  Female Gender  Average Difficulty  Number of Online Ratings
3                0.0            0.0            0.625            0.012887
5                1.0            0.0            0.575            0.043814
21               1.0            0.0            0.775            0.012887
25               0.0            1.0            0.575            0.028351
27               0.0            1.0            0.200            0.007732

```

```

Received a Pepper  Proportion Take Again
3                1.0            0.822917
5                0.0            0.541667
21               0.0            0.822917
25               1.0            0.822917
27               0.0            0.822917

```

below is the summary of this normalized multiple regression model (Not OLS Regression Results)

```

=====
Dep. Variable:      Average Rating      R-squared:
Model:              OLS                Adj. R-squared:
Method:             Least Squares       F-statistic:
Date:               Mon, 27 Oct 2025     Prob (F-statistic):
Time:               19:24:52            Log-Likelihood:
No. Observations:   25368              AIC:
Df Residuals:       25360              BIC:
Df Model:           7
Covariance Type:    nonrobust
=====

```

	coef	std err	t	P> t
const	3.3898	0.025	137.972	0.000
Male Gender	0.1061	0.009	11.355	0.000
Female Gender	0.0585	0.010	6.078	0.000
Average Difficulty	-2.1439	0.021	-101.882	0.000
Number of Ratings	1.4085	0.137	10.274	0.000
Received a Pepper	0.4968	0.009	58.174	0.000
Number of Online Ratings	-0.1366	0.048	-2.831	0.005
Proportion Take Again	1.5068	0.023	64.547	0.000

```

=====
Omnibus:            1971.619      Durbin-Watson:
Prob (Omnibus) =    0.000      Prob (DW) =

```

Prob(Omnibus):	0.000	Jarque-Bera (JB):
Skew:	-0.658	Prob(JB):
Kurtosis:	3.920	Cond. No.

=====

Notes:

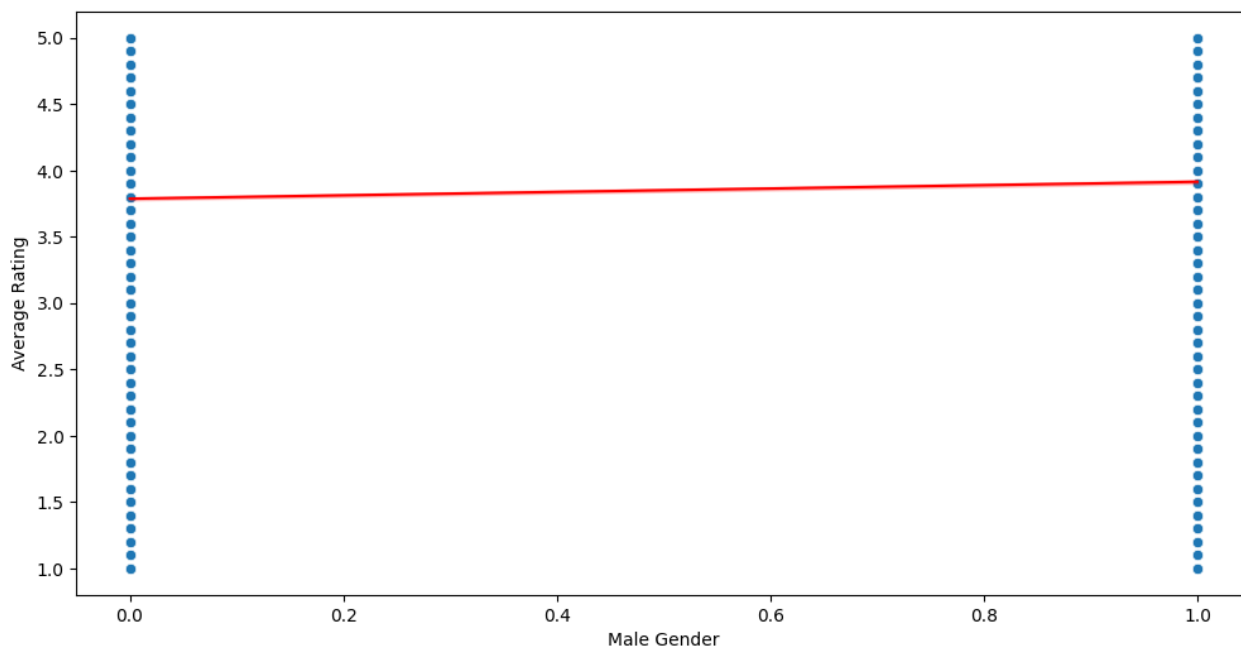
[1] Standard Errors assume that the covariance matrix of the errors is

```
from sklearn.linear_model import Ridge
ridge_model = Ridge(alpha=100)
ridge_model.fit(X, y)
y_pred = ridge_model.predict(X)
r_squared = ridge_model.score(X, y)
# rmse = mean_squared_error(y, y_pred, squared=False)
ridge_coefficients = ridge_model.coef_
ridge_intercept = ridge_model.intercept_
print(f"Regression Equation After Lasso: y = {ridge_intercept:.4f} + '
```

Regression Equation After Lasso: y = 3.4076 + 0.0 * const + 0.10922219:

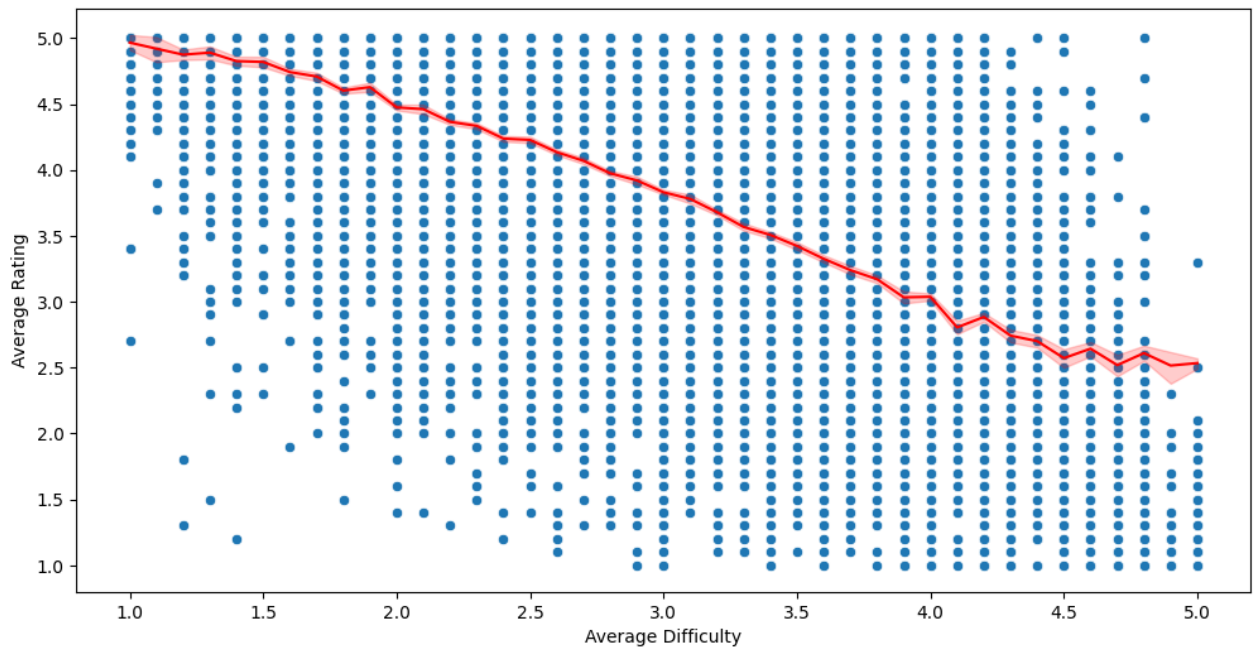
```
# here, we make two scatter plots to visualize the effect of male gender
# and we find out that though there is a positive relationship, but relatively weak
plt.figure(figsize=(12, 6))
sns.scatterplot(x='Male Gender', y='Average Rating', data=merged_data)
sns.lineplot(x=merged_data_cleaned["Male Gender"], y=model.fittedvalues)

plt.show()
```



```
# In fact, the Average Difficulty has a stronger negative relationship
# as the Average Difficulty increases, the average rating goes down.
plt.figure(figsize=(12, 6))
sns.scatterplot(x='Average Difficulty', y='Average Rating', data=merged_data)
sns.lineplot(x=merged_data_cleaned["Average Difficulty"], y=model.fit_predict(merged_data_cleaned["Average Difficulty"]))

plt.show()
```



```
#As for the impact of gender: the score of male professors is indeed :
#but the absolute value of the regression coefficient is very small;
#Grades are driven more by the difficulty of the course, the number of
```

✓ Analysis 2

Is there an effect of experience on the quality of teaching? You can operationalize quality with average rating and use the number of ratings as an imperfect – but available – proxy for experience. Again, a significance test is probably a good idea.

```
#####Analysis 2
print()
print("Analysis 2")
print()
num_median = merged_data_cleaned["Number of Ratings"].median()
high_num_ratings = merged_data_cleaned[merged_data_cleaned["Number of Ratings"] > num_median]
low_num_ratings = merged_data_cleaned[merged_data_cleaned["Number of Ratings"] <= num_median]
# Test Normality of distribution
shapiro_test_high = shapiro(high_num_ratings)
shapiro_test_low = shapiro(low_num_ratings)
print(shapiro_test_high.pvalue, shapiro_test_low.pvalue)
```

Analysis 2

```
8.375907528527551e-64 1.7562812667206124e-64
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:100: FutureWarning:
    res = hypotest_fun_out(*samples, **kwds)
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:100: FutureWarning:
    res = hypotest_fun_out(*samples, **kwds)
```

```
# Visualize the distribution, and we can see both group are not normal
plt.figure(figsize=(12, 6))
sns.kdeplot(high_num_ratings, label="High Num Ratings", shade=True, color="red")
sns.kdeplot(low_num_ratings, label="Low Num Ratings", shade=True, color="blue")
plt.title("Distribution of Average Rating")
plt.legend()
plt.show()
```

```
/tmp/ipython-input-3754329928.py:3: FutureWarning:
```

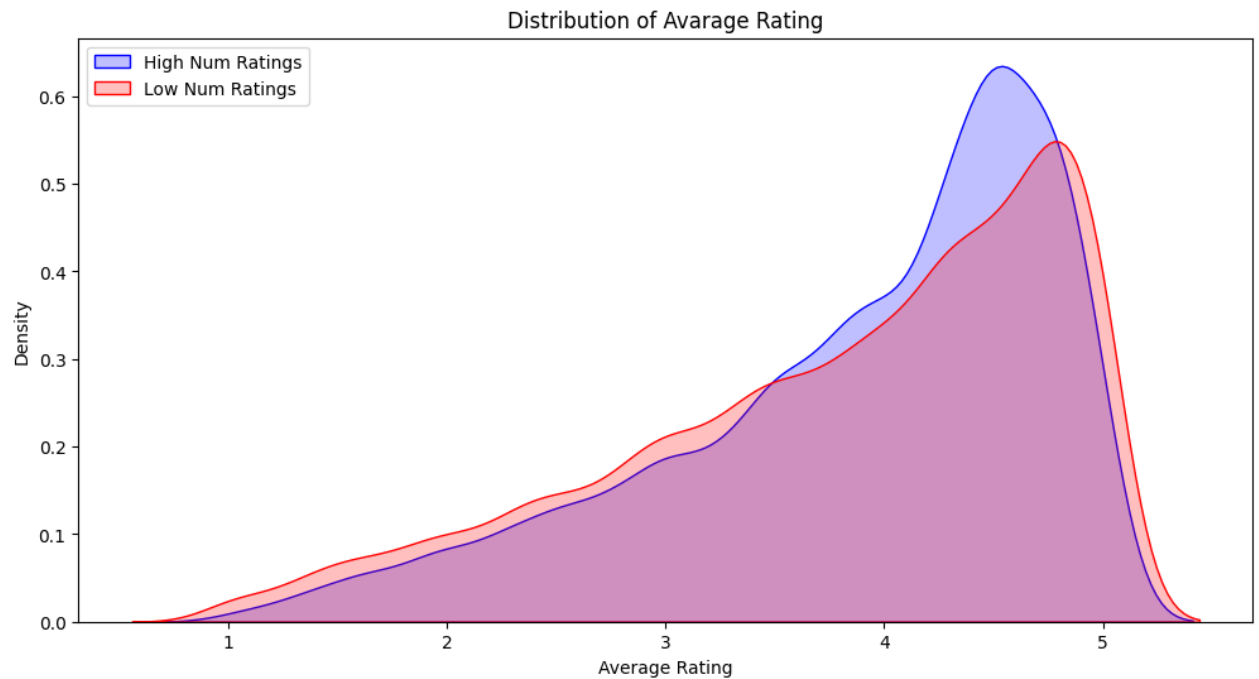
```
`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.
```

```
sns.kdeplot(high_num_ratings, label="High Num Ratings", shade=True, color="red")
/tmp/ipython-input-3754329928.py:4: FutureWarning:
```

```
`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.
```

THIS WILL BECOME AN ERROR IN SEABORN V0.11.0, PLEASE UPDATE YOUR CODE.

```
sns.kdeplot(low_num_ratings, label="Low Num Ratings", shade=True, col
```



```
#find out that there is a true difference of two group since p-value.  
#makes the avergae rating higher, but we still need to test the streng  
from scipy.stats import mannwhitneyu  
u_stats, u_p_value = mannwhitneyu(high_num_ratings, low_num_ratings)  
print()  
print("Question 2 Mann-Whitney U-test", "Q2 U stats",u_stats, "Q2 U_p.  
print()
```

Question 2 Mann-Whitney U-test Q2 U stats 82000665.0 Q2 U_p_value 0.000

```

# Here we use this regression model to see the strength of experience (
# and we find out that though it poses a positive effect as the experie
# the effect is very small.
import statsmodels.api as sm
X = merged_data_cleaned['Number of Ratings']
y = merged_data_cleaned["Average Rating"]
X = sm.add_constant(X)
model = sm.OLS(y, X).fit()
print(model.summary())

```

OLS Regression Results

```

=====
Dep. Variable:          Average Rating    R-squared:
Model:                  OLS              Adj. R-squared:
Method:                 Least Squares    F-statistic:
Date:                  Mon, 27 Oct 2025   Prob (F-statistic):
Time:                  19:25:27          Log-Likelihood:
No. Observations:      25368            AIC:
Df Residuals:          25366            BIC:
Df Model:               1
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.0
const	3.7869	0.008	454.711	0.000	3.
Number of Ratings	0.0048	0.001	9.227	0.000	0.0

```

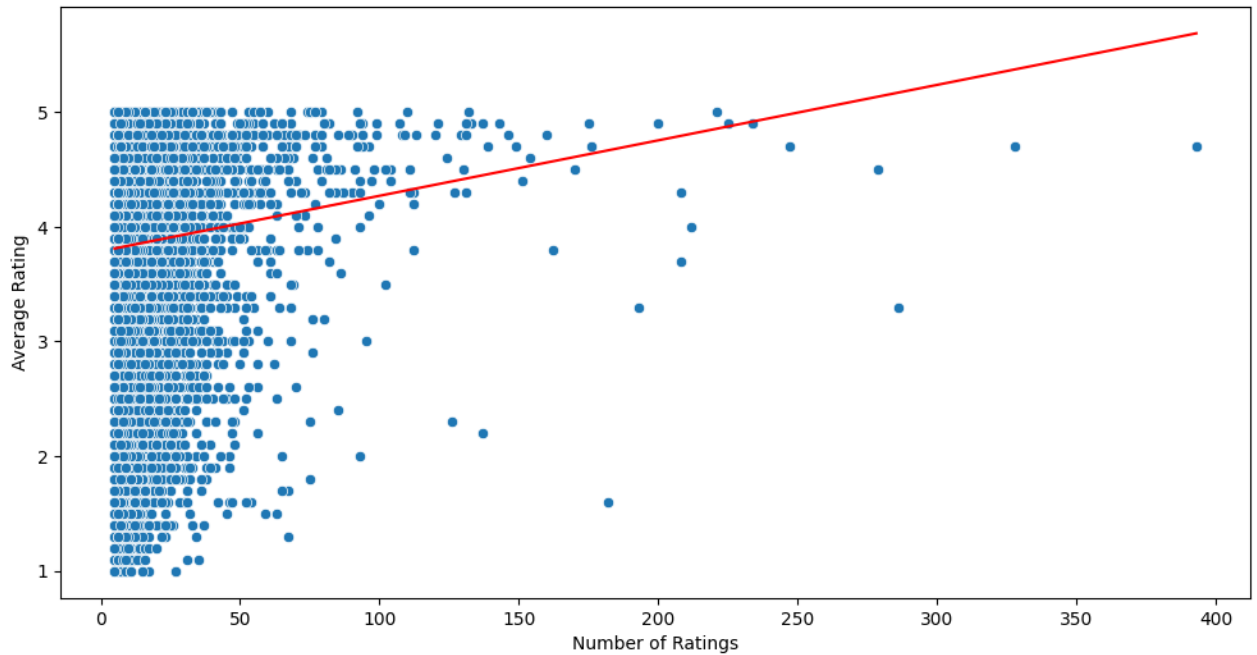
=====
Omnibus:                2399.111    Durbin-Watson:
Prob(Omnibus):           0.000      Jarque-Bera (JB):
Skew:                   -0.864      Prob(JB):
Kurtosis:               2.950      Cond. No.
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is

```
#Shows from the graph, though the regression line looks steep, there is
#The Number of Ratings increases about 100, the Average rating increase
plt.figure(figsize=(12, 6))
sns.scatterplot(x='Number of Ratings', y='Average Rating', data=merged_
sns.lineplot(x=merged_data_cleaned['Number of Ratings'], y=model.fitted
plt.show()
```



✓ Analysis 3

What is the relationship between average rating and average difficulty?

```

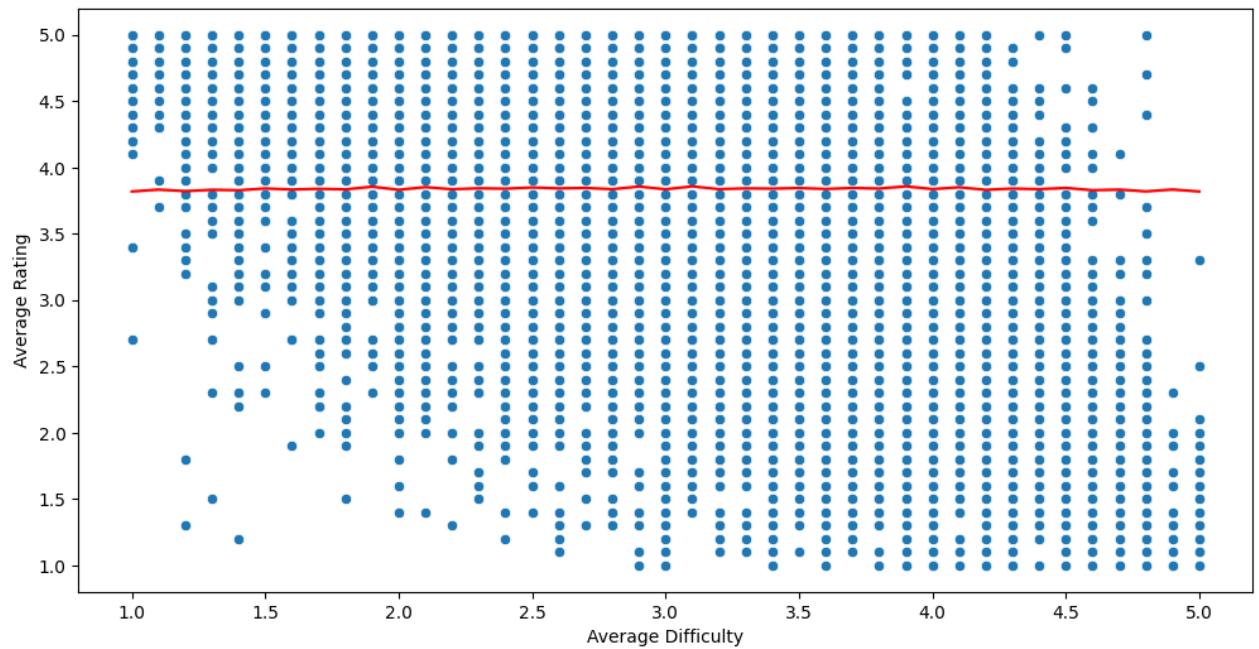
#Analysis 3
print()
print("Analysis 3")
print()
#First, I used a Linregress model to see how much variance in the ave
from scipy.stats import linregress
slope, intercept, r_value, p_value, std_err = linregress(merged_data_
print({
    "Slope": slope,
    "Intercept": intercept,
    "R_value": r_value,
    "R-squared": r_value**2,
    "P-value": p_value,
    "Standard Error": std_err
})

```

Analysis 3

```
{'Slope': np.float64(-0.5269918767745604), 'Intercept': np.float64(4.9!
```

```
plt.figure(figsize=(12, 6))
sns.scatterplot(x='Average Difficulty', y="Average Rating", data=merged_data_cleaned)
sns.lineplot(x=merged_data_cleaned["Average Difficulty"], y=model.fit_predict(merged_data_cleaned["Average Difficulty"]))
plt.show()
```



```
#R-squared is only 0.3832, which is small, then I planned to use a spearmanr
from scipy.stats import spearmanr
spearman_corr, spearman_p_value = spearmanr(merged_data_cleaned["Average"])

print({
    "Spearman Correlation": spearman_corr,
    "P-value": spearman_p_value
})
#Conclusion
#We say the Spearman correlation coefficient (-0.6027) indicates a moderate
#of moderate strength, and the p_value (0.0) indicates a statistically
#But the correlation of the Negative linear relationship is still a lot
```

```
{'Spearman Correlation': np.float64(-0.6016691560542752), 'P-value': np.float64(0.0)}
```

✓ Analysis 4

Do professors who teach a lot of classes in the online modality receive higher or lower ratings than those who don't? Hint: A significance test might be a good idea, but you need to think of a creative but suitable way to split the data.

```

#Analysis 4:
print()
print("Analysis 4")
print()
#To determine whether there is a difference in average rating between
#who teach a low online modality or not teaching a lot online modality
#data by median into two group, and this ensures same number of profes

from scipy import stats
# Find median value for "Number of Online Ratings"
median_online_ratings = merged_data_cleaned["Number of Online Ratings"]
#Notice, not equal size. the median is 0.
print(median_online_ratings)
# then I decide to test the percentage of professor who received 1,2,4
low_threshold = merged_data_cleaned["Number of Online Ratings"].quantile(0.5)
# still low threshold is 0 too, so we set low experience group as number of online ratings less than 2
print(low_threshold)
high_threshold = 2
total_professors = len(merged_data_cleaned)
professors_with_online_ratings = len(merged_data_cleaned[merged_data_cleaned["Number of Online Ratings"] > 0])
percentage_with_online_ratings = (professors_with_online_ratings / total_professors) * 100
print(f"Percentage of professors with online ratings: {percentage_with_online_ratings}%")

```

Analysis 4

0.0

0.0

Percentage of professors with online ratings: 10.59%

```

# Split into two groups: low online modality and high online modality
low_online_group = merged_data_cleaned[merged_data_cleaned["Number of Online Ratings"] < 2]
high_online_group = merged_data_cleaned[merged_data_cleaned["Number of Online Ratings"] > 2]

```

```

# Test for normality in both groups
shapiro_test_low = stats.shapiro(low_online_group["Average Rating"])
shapiro_test_high = stats.shapiro(high_online_group["Average Rating"])
print(shapiro_test_low.pvalue, shapiro_test_high.pvalue)
#They are not normally distributed, thus we need to use an nonparametric test
#Since we are not interested in how two group are distributed, we use

```

```

1.1862634447143301e-73 1.775452880820878e-35
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py:100: RuntimeWarning:
  res = hypotest_fun_out(*samples, **kws)

```

```

#make a distribution graph of density of Average rating and Online Modality

```

```
plt.figure(figsize=(10, 6))
sns.kdeplot(low_online_group["Average Rating"], label="Low Online Modality")
sns.kdeplot(high_online_group["Average Rating"], label="High Online Modality")
plt.title("Distribution of Average Ratings by Online Teaching Modality")
plt.xlabel("Average Rating")
plt.ylabel("Density")
plt.legend()
plt.show()
```

/tmp/ipython-input-2821835969.py:3: FutureWarning:

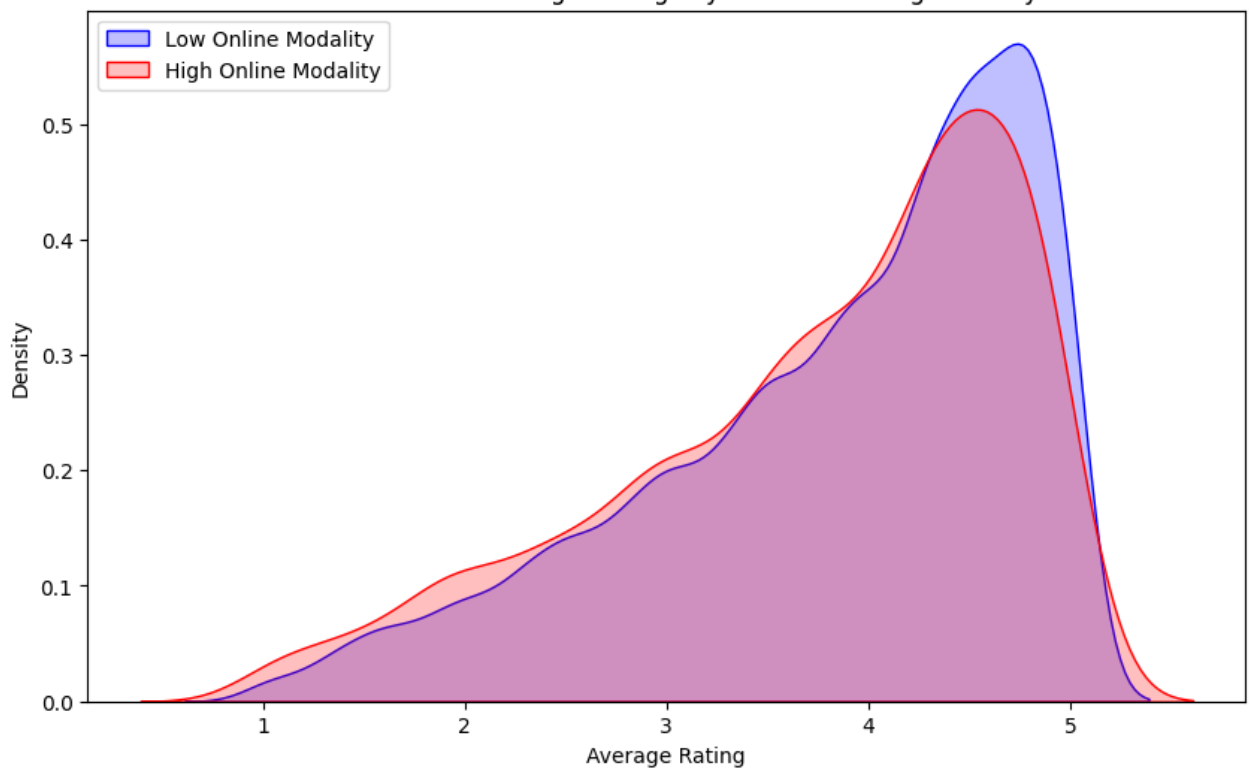
`shade` is now deprecated in favor of `fill`; setting `fill=True`. This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(low_online_group["Average Rating"], label="Low Online Modality")
/tmp/ipython-input-2821835969.py:4: FutureWarning:
```

`shade` is now deprecated in favor of `fill`; setting `fill=True`. This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(high_online_group["Average Rating"], label="High Online Modality")
```

Distribution of Average Ratings by Online Teaching Modality



```
#test the proportion of professor who received the online rating base
#Finding the threshold for "Lot of online modality"
print(low_online_group["Number of Online Ratings"].median())
print(high_online_group["Number of Online Ratings"].median())
#Median Average rating of low online group"
print("Median Average rating of low online group",low_online_group["A
#Median Average rating of High online group"
print("Median Average rating of High online group",high_online_group['
u_stats, u_p_value = stats.mannwhitneyu(low_online_group["Average Rat
print("Mann-Whitney U-test", "u_stats:",u_stats,"u_p_value:", u_p_va
#We can see that the p_value is extremelly small, thus we can drop the
#who teach a lower proportion of their classes online receive higher
```

```
0.0
3.0
Median Average rating of low online group 4.1
Median Average rating of High online group 4.0
Mann-Whitney U-test u_stats: 29199517.5 u_p_value: 8.911466773307417e-
```

✓ Analysis 5

What is the relationship between the average rating and the proportion of people who would take the class the professor teaches again?

```

print()
print("Analysis 5")
print()
merged_data_5 = merged_data.dropna(subset=["Average Rating", "Average
    "Received a Pepper", "Number of Online Ratings", "Major/Field", "I
merged_data_5 = merged_data_5.dropna(subset=["Proportion Take Again"])

X = merged_data_5["Proportion Take Again"]
y = merged_data_5["Average Rating"]

slope, intercept, r_value, p_value, std_err = linregress(X, y)

print("slope:",slope, "intercept:",intercept, "r_value:",r_value, "r_

```

Analysis 5

slope: 0.029785869296643768 intercept: 1.6599849590247624 r_value: 0.88

```

from scipy.stats import pearsonr
correlation, p_value = pearsonr(merged_data_5["Average Rating"], merge
print("correlation:",correlation, "p_value:",p_value)

```

correlation: 0.8803535424363584 p_value: 0.0

```

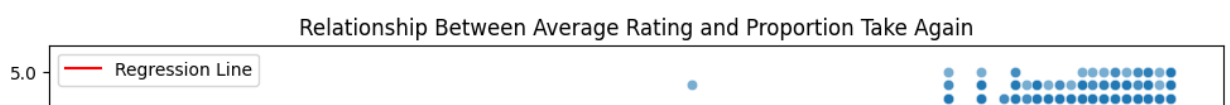
plt.figure(figsize=(12, 6))
# Scatter plot of data points
sns.scatterplot(x="Proportion Take Again", y="Average Rating", data=me

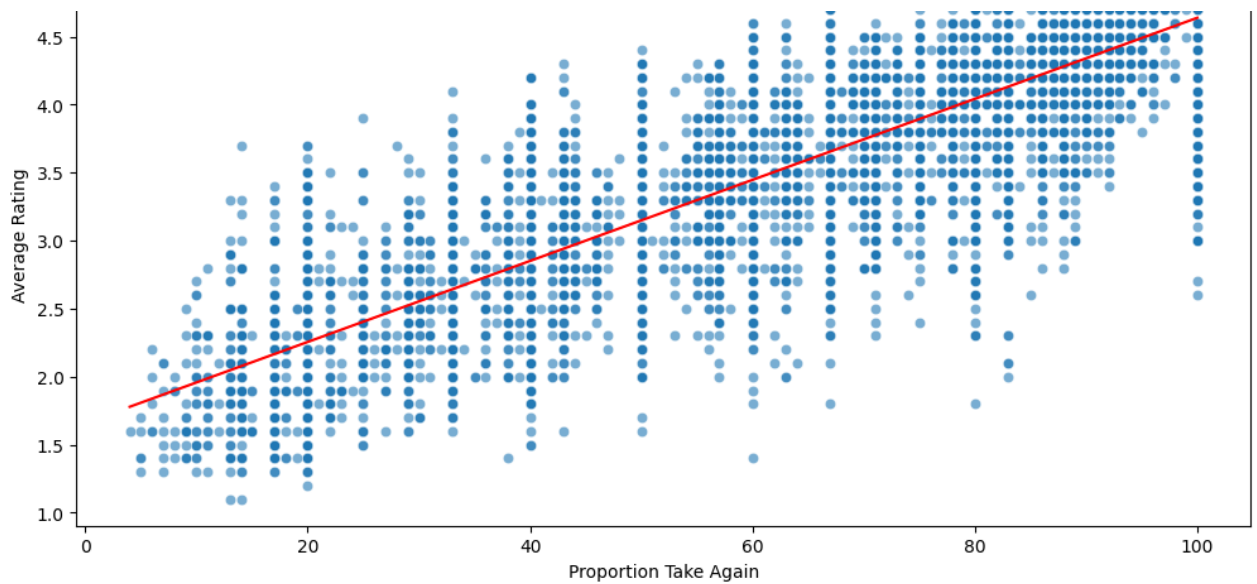
# Regression line
sns.lineplot(x=merged_data_5["Proportion Take Again"],
             y=(slope * merged_data_5["Proportion Take Again"] + inte
             color="red", label="Regression Line")

# Add labels and title
plt.title("Relationship Between Average Rating and Proportion Take Ag
plt.xlabel("Proportion Take Again")
plt.ylabel("Average Rating")
plt.legend()

plt.show()

```





✓ Analysis 6

Do professors who are “hot” receive higher ratings than those who are not? Again, a significance test is indicated.

```
print()  
print("Analysis 6")  
print()
```

Analysis 6

```
#Split the data to Professor who are "Hot" (Received a Pepper) and Pro
hot_group = merged_data_cleaned[merged_data_cleaned["Received a Pepper"]
not_hot_group = merged_data_cleaned[merged_data_cleaned["Received a Pe
#Test the distribution
shapiro_hot = stats.shapiro(hot_group)
shapiro_not_hot = stats.shapiro(not_hot_group)

print(shapiro_hot.pvalue, shapiro_not_hot.pvalue)
```

```
9.477384543093599e-68 4.67978722924884e-51
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py
res = hypotest_fun_out(*samples, **kws)
/usr/local/lib/python3.12/dist-packages/scipy/stats/_axis_nan_policy.py
res = hypotest_fun_out(*samples, **kws)
```

```
plt.figure(figsize=(12, 6))
# plot for "hot" professors
sns.kdeplot(hot_group, label="Hot Professors", shade=True, color="orange")
# plot for "not hot" professors
sns.kdeplot(not_hot_group, label="Not Hot Professors", shade=True, color="blue")
plt.title("Distribution of Average Ratings for Hot vs. Not Hot Profes")
plt.xlabel("Average Rating")
plt.ylabel("Density")
plt.legend()
plt.show()
```

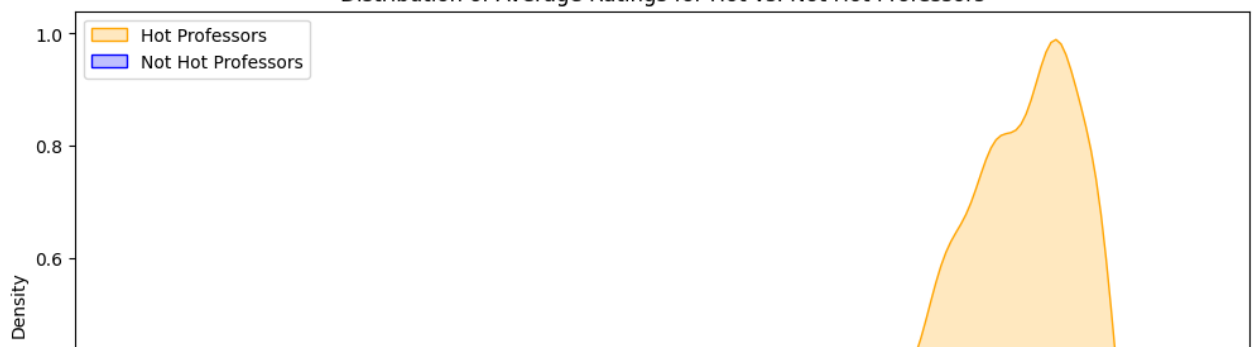
/tmp/ipython-input-3149484334.py:3: FutureWarning:

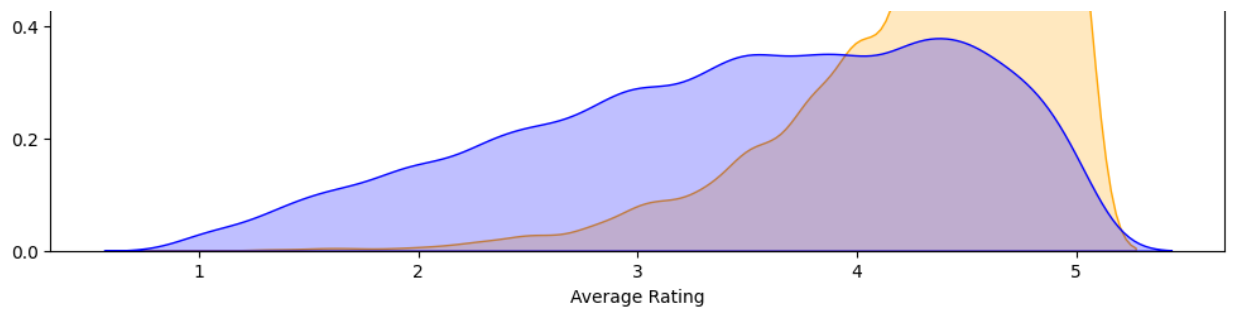
`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(hot_group, label="Hot Professors", shade=True, color="orange")
/tmp/ipython-input-3149484334.py:5: FutureWarning:
```

`shade` is now deprecated in favor of `fill`; setting `fill=True`.
This will become an error in seaborn v0.14.0; please update your code.

```
sns.kdeplot(not_hot_group, label="Not Hot Professors", shade=True, color="blue")
Distribution of Average Ratings for Hot vs. Not Hot Professors
```





```
u_stat, u_p_value = stats.mannwhitneyu(hot_group, not_hot_group)
print("Mann-Whitney U-test", "Q6_u_stat:",u_stat, "Q6_u_p_value:", u_
print("Median of the hot group:",hot_group.median())
print("Median of the Not hot group:",not_hot_group.median())
```

```
Mann-Whitney U-test Q6_u_stat: 121910581.5 Q6_u_p_value: 0.0
Median of the hot group: 4.5
Median of the Not hot group: 3.6
```

✓ Analysis 7

Build a regression model predicting average rating from difficulty (only). Make sure to include the R2 and RMSE of this model.

```

#Analysis 7
print()
print("Analysis 7")
print()
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split

X = merged_data_5[["Average Difficulty"]].values
y = merged_data_5["Average Rating"].values

```

Analysis 7

```

#Split the data to ensure not over fitt.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
# Train the regression model on the training set
model = LinearRegression()
model.fit(X_train, y_train)
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)

```

```

r_squared_train = model.score(X_train, y_train)
mse_train = mean_squared_error(y_train, y_pred_train)
rmse_train = np.sqrt(mse_train)

```

```

r_squared_test = model.score(X_test, y_test)
mse_test = mean_squared_error(y_test, y_pred_test)
rmse_test = np.sqrt(mse_test)

```

```

intercept = model.intercept_
slope = model.coef_[0]
print(f"Regression Equation: y = {intercept:.4f} + {slope:.4f} * x")
print(f"R-squared: training : {r_squared_train}, test : {r_squared_test}")
print(f"RMSE: train : {rmse_train}, test : {rmse_test}")

```

```

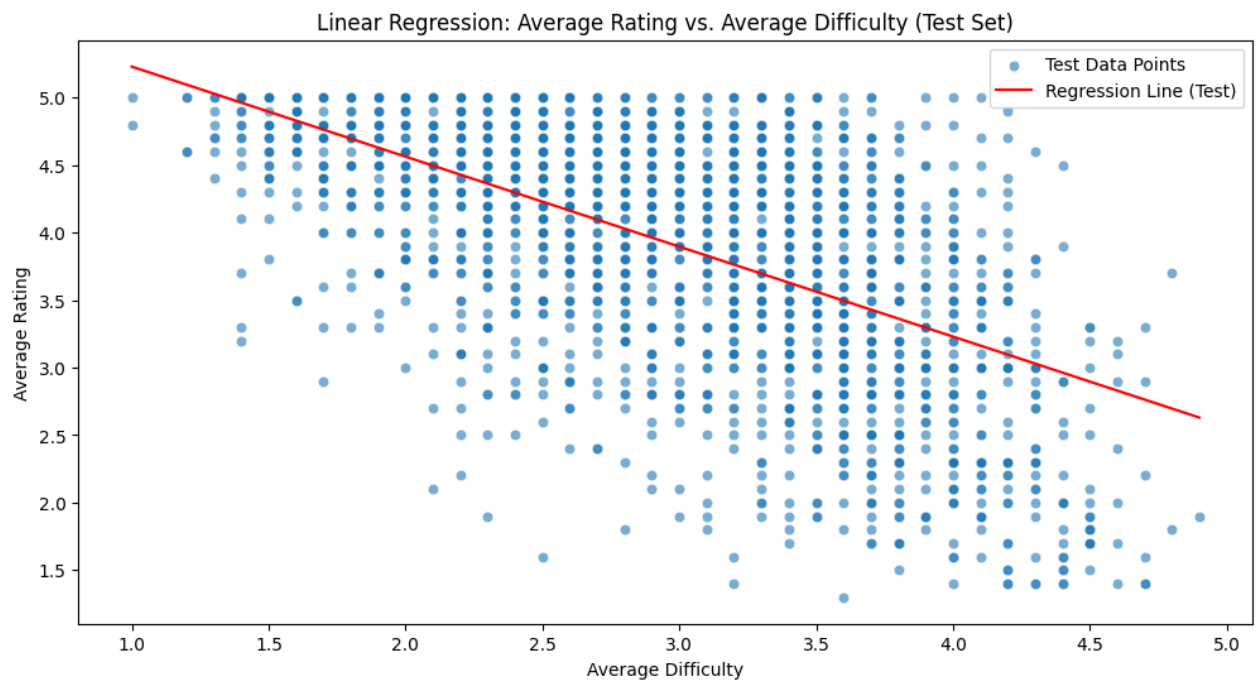
Regression Equation: y = 5.8946 + -0.6663 * x
R-squared: training : 0.3414794824354239, test : 0.3453726137189621
RMSE: train : 0.6859573448171066, test : 0.687545104370505

```

```

plt.figure(figsize=(12, 6))
# Scatter plot of actual test data
sns.scatterplot(x=X_test.flatten(), y=y_test, alpha=0.6, label="Test Data Points")
# Regression line
sns.lineplot(x=X_test.flatten(), y=y_pred_test, color="red", label="Regression Line (Test)")
# Add title and labels
plt.title("Linear Regression: Average Rating vs. Average Difficulty (Test Set)")
plt.xlabel("Average Difficulty")
plt.ylabel("Average Rating")
plt.legend()
plt.show()
## the r-square of single factor is too low, the explanation power is

```



✓ Analysis 8

Build a regression model predicting average rating from all available factors. Make sure to include the R2 and RMSE of this model. Comment on how this model compares to the “difficulty only” model and on individual betas. Hint: Make sure to address collinearity concerns.

```
# Analysis 8
print()
print("Analysis 8")
print()
y = merged_data_5["Average Rating"]
X = merged_data_5[["Average Difficulty", "Number of Ratings",
    "Received a Pepper", "Proportion Take Again", "Number of Online Ratings",
    "Male Gender", "Female Gender"]]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)
model = LinearRegression()
model.fit(X_train, y_train)
y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)
r_squared_train = model.score(X_train, y_train)
mse_train = mean_squared_error(y_train, y_pred_train)
rmse_train = np.sqrt(mse_train)
r_squared_test = model.score(X_test, y_test)
mse_test = mean_squared_error(y_test, y_pred_test)
rmse_test = np.sqrt(mse_test)

intercept = model.intercept_
slopes = model.coef_
print(f"Regression Equation: y = {intercept:.4f} + " + " + ".join([f"-{slope:.4f} * {variable}" for variable, slope in zip(X.columns, slopes)]) + ".")
print(f"R-squared: training : {r_squared_train}, test : {r_squared_test}")
print(f"RMSE: train : {rmse_train}, test : {rmse_test}")
```

Analysis 8

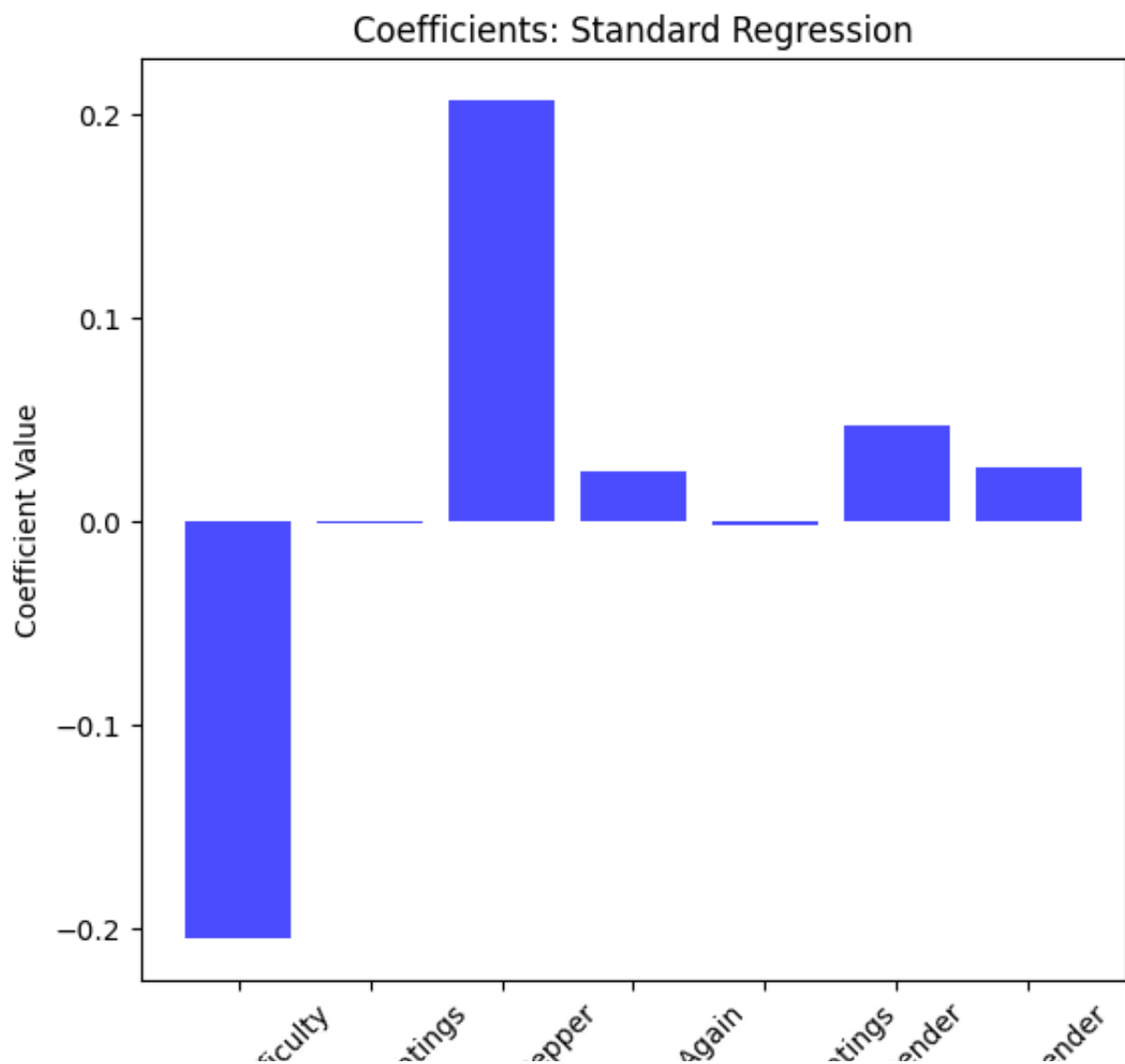
Regression Equation: $y = 2.5343 + -0.20459215002616857 * \text{Average Difficulty} + 0.0000000000000000 * \text{Number of Ratings} + 0.0000000000000000 * \text{Received a Pepper} + 0.0000000000000000 * \text{Proportion Take Again} + 0.0000000000000000 * \text{Number of Online Ratings} + 0.0000000000000000 * \text{Male Gender} + 0.0000000000000000 * \text{Female Gender}$
R-squared: training : 0.8099508966483329, test : 0.8105314551764243
RMSE: train : 0.36831872666463006, test : 0.3706311680459302

```
#Plot the coefficient bar graph before dealing with collinearity
variables = ["Average Difficulty", "Number of Ratings", "Received a Pepper",
    "Proportion Take Again", "Number of Online Ratings", "Male Gender", "Female Gender"]
standard_coefficients = slopes
```

```
# Plot comparison of coefficients
plt.figure(figsize=(14, 6))

# Standard Regression Coefficients
plt.subplot(1, 2, 1)
plt.bar(variables, slopes, alpha=0.7, color='blue')
plt.title("Coefficients: Standard Regression")
plt.xlabel("Independent Variables")
plt.ylabel("Coefficient Value")
plt.xticks(rotation=45)
```

```
([0, 1, 2, 3, 4, 5, 6],
 [Text(0, 0, 'Average Difficulty'),
  Text(1, 0, 'Number of Ratings'),
  Text(2, 0, 'Received a Pepper'),
  Text(3, 0, 'Proportion Take Again'),
  Text(4, 0, 'Number of Online Ratings'),
  Text(5, 0, 'Male Gender'),
  Text(6, 0, 'Female Gender')])
```



Average Diffi

Number of Ra

Received a Pe

Proportion Take r

Number of Online Ra

Male Ge

Female Ge

Independent Variables

```
# Colinearity:
import pandas as pd
correlation_matrix = pd.DataFrame(X).corr()
print(correlation_matrix)
```

	Average Difficulty	Number of Ratings	\
Average Difficulty	1.000000	0.002798	
Number of Ratings	0.002798	1.000000	
Received a Pepper	-0.246359	0.105538	
Proportion Take Again	-0.517074	0.055805	
Number of Online Ratings	-0.021126	0.091293	
Male Gender	-0.021091	0.021866	
Female Gender	-0.035559	-0.043859	

	Received a Pepper	Proportion Take Again	\
Average Difficulty	-0.246359	-0.517074	
Number of Ratings	0.105538	0.055805	
Received a Pepper	1.000000	0.442400	
Proportion Take Again	0.442400	1.000000	
Number of Online Ratings	0.008880	-0.013181	
Male Gender	-0.016395	0.070023	
Female Gender	0.027505	-0.008060	

	Number of Online Ratings	Male Gender	Female Gender
Average Difficulty	-0.021126	-0.021091	-0.035559
Number of Ratings	0.091293	0.021866	-0.043859
Received a Pepper	0.008880	-0.016395	0.027505
Proportion Take Again	-0.013181	0.070023	-0.008060
Number of Online Ratings	1.000000	-0.030235	0.067672
Male Gender	-0.030235	1.000000	-0.521513
Female Gender	0.067672	-0.521513	1.000000

```
#Since we consider all predictors are potentially important, I do not  
from sklearn.linear_model import Ridge
```

```
ridge_model = Ridge(alpha=100)  
ridge_model.fit(X, y)
```

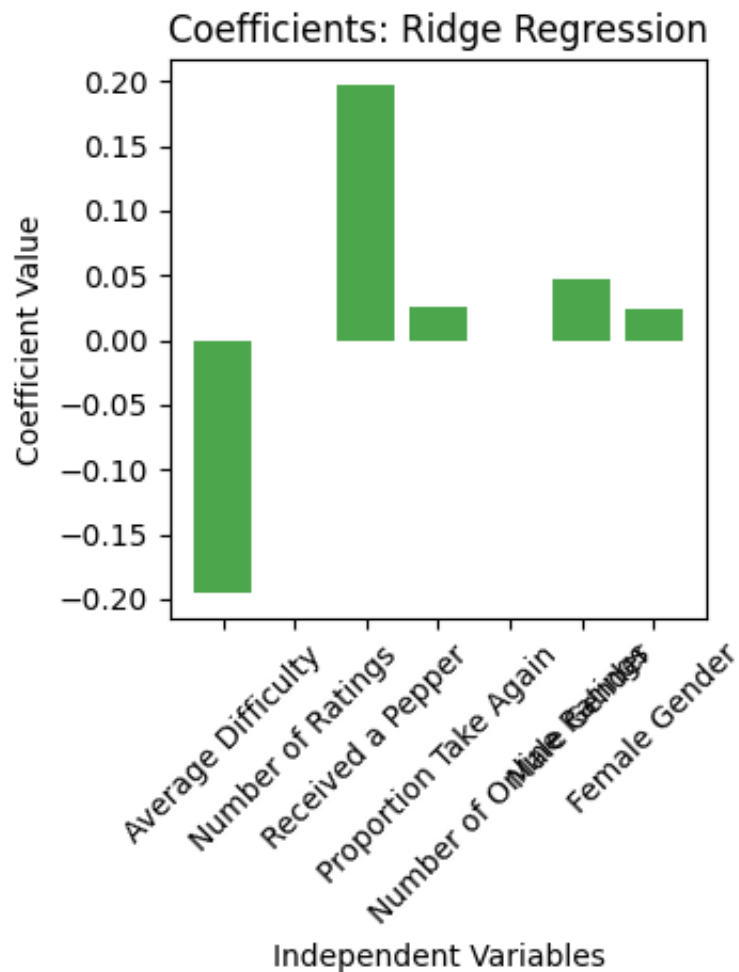
```
y_pred = ridge_model.predict(X)  
r_squared = ridge_model.score(X, y)  
mse = mean_squared_error(y, y_pred)  
rmse=np.sqrt(mse)  
ridge_coefficients = ridge_model.coef_  
ridge_intercept = ridge_model.intercept_
```

```
print(f"Ridge Regression Equation: y = {ridge_intercept:.4f} + " + " -  
print(f"R-squared: training : {r_squared_train}, test : {r_squared_te:  
print(f"RMSE: train : {rmse_train}, test : {rmse_test}")
```

```
Ridge Regression Equation: y = 2.4829 + -0.19556158768461207 * Average  
R-squared: training : 0.8099508966483329, test : 0.8105314551764243  
RMSE: train : 0.36831872666463006, test : 0.3706311680459302
```

```
# Ridge Regression Coefficients
plt.subplot(1, 2, 2)
plt.bar(variables, ridge_coefficients, alpha=0.7, color='green')
plt.title("Coefficients: Ridge Regression")
plt.xlabel("Independent Variables")
plt.ylabel("Coefficient Value")
plt.xticks(rotation=45)

# Adjust layout for better visualization
plt.tight_layout()
plt.show()
```



```

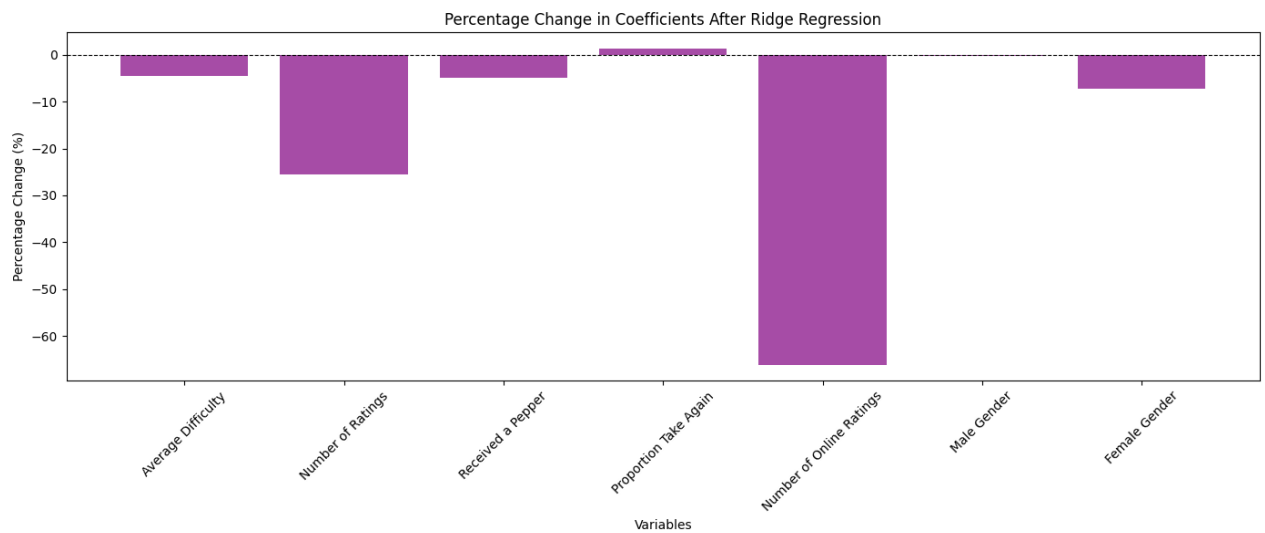
# percentage of change from standard coefficients to ridge coefficient
# Coefficients from Standard Regression
standard_coefficients = slopes
# Coefficients from Ridge Regression
ridge_coefficients = ridge_model.coef_
variables = ["Average Difficulty", "Number of Ratings", "Received a Pepper",
             "Proportion Take Again", "Number of Online Ratings", "Male Gender", "Female Gender"]
# Calculate Percentage Change
percentage_change = ((ridge_coefficients - standard_coefficients) / standard_coefficients) * 100
# Create DataFrame for visualization
coefficients_comparison = pd.DataFrame({
    "Variable": variables,
    "Standard Coefficients": standard_coefficients,
    "Ridge Coefficients": ridge_coefficients,
    "Percentage Change (%)": percentage_change
})
print(coefficients_comparison)

```

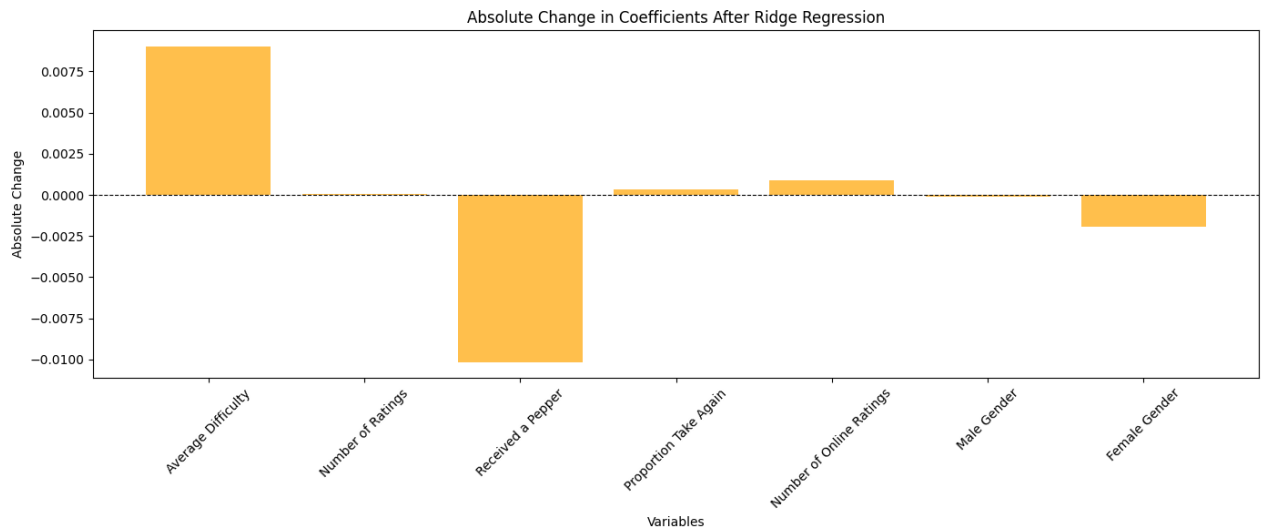
	Variable	Standard Coefficients	Ridge Coefficients
0	Average Difficulty	-0.204592	-0.195562
1	Number of Ratings	-0.000263	-0.000196
2	Received a Pepper	0.206937	0.196773
3	Proportion Take Again	0.024660	0.025008
4	Number of Online Ratings	-0.001305	-0.000441
5	Male Gender	0.047744	0.047623
6	Female Gender	0.026792	0.024840

	Percentage Change (%)
0	-4.413934
1	-25.457239
2	-4.911229
3	1.409336
4	-66.179482
5	-0.252883
6	-7.284320

```
# Plot Percentage Change
plt.figure(figsize=(14, 6))
plt.bar(coefficients_comparison["Variable"], coefficients_comparison['
plt.title("Percentage Change in Coefficients After Ridge Regression")
plt.xlabel("Variables")
plt.ylabel("Percentage Change (%)")
plt.xticks(rotation=45)
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
plt.tight_layout()
plt.show()
```



```
# Plot Absolute Change
absolute_change = ridge_coefficients - standard_coefficients
plt.figure(figsize=(14, 6))
plt.bar(coefficients_comparison["Variable"], absolute_change, color='orange')
plt.title("Absolute Change in Coefficients After Ridge Regression")
plt.xlabel("Variables")
plt.ylabel("Absolute Change")
plt.xticks(rotation=45)
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
plt.tight_layout()
plt.show()
```



✓ Analysis 9

Build a classification model that predicts whether a professor receives a “pepper” from average rating only. Make sure to include quality metrics such as AU(RO)C and also address class imbalances.

```

#Analysis 9
print()
print()
print("Analysis 9")
print()
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score, roc_curve, confusion_matrix
from imblearn.over_sampling import SMOTE
X = merged_data_5[['Average Rating']]
y = merged_data_5["Received a Pepper"]
#Checked distribution to see is it imbalanced (result, no pepper: Pepper)
class_distribution = y.value_counts(normalize=True)
print(class_distribution)

```

Analysis 9

```

Received a Pepper
0.0    0.534046
1.0    0.465954
Name: proportion, dtype: float64

```

```

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y
)
# Use SMOTE to resample equal sized sample (balancing)
smote = SMOTE()
X_train_resample, y_train_resample = smote.fit_resample(X_train, y_train)
print(y_train_resample.value_counts(normalize=True))
# use logistic regression, the simplest one, see performance (ROC AUC)
logistic_model = LogisticRegression()
# logistic_model = LogisticRegression(class_weight="balanced")
logistic_model.fit(X_train_resample, y_train_resample)
y_pred_train = logistic_model.predict(X_train_resample)
y_pred = logistic_model.predict(X_test)
y_pred_prob = logistic_model.predict_proba(X_test)[:, 1]
y_pred_prob_train = logistic_model.predict_proba(X_train_resample)[:, 1]

```

```

Received a Pepper
1.0    0.5
0.0    0.5
Name: proportion, dtype: float64

```

```

roc_auc = roc_auc_score(y_test, y_pred_prob)
roc_auc_train = roc_auc_score(y_train_resample, y_pred_prob_train)
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
conf_matrix = confusion_matrix(y_test, y_pred)
classification_metrics = classification_report(y_test, y_pred, output_

# High ROC AUC, model have good performance, train and test ROC AUC is
print({
    "Class Distribution" : class_distribution.to_dict(),
    "Train ROC AUC" : roc_auc_train,
    "ROC AUC" : roc_auc,
    "Confusion Matrix" : conf_matrix.tolist(),
    "Classification Metrics" : classification_report
})

{'Class Distribution': {0.0: 0.5340460526315789, 1.0: 0.465953947368421}

```

```

print("Confusion Metrix Q9:")
print(conf_matrix)
print("Classification_metrics Q9")
print(classification_report(y_test, y_pred))

```

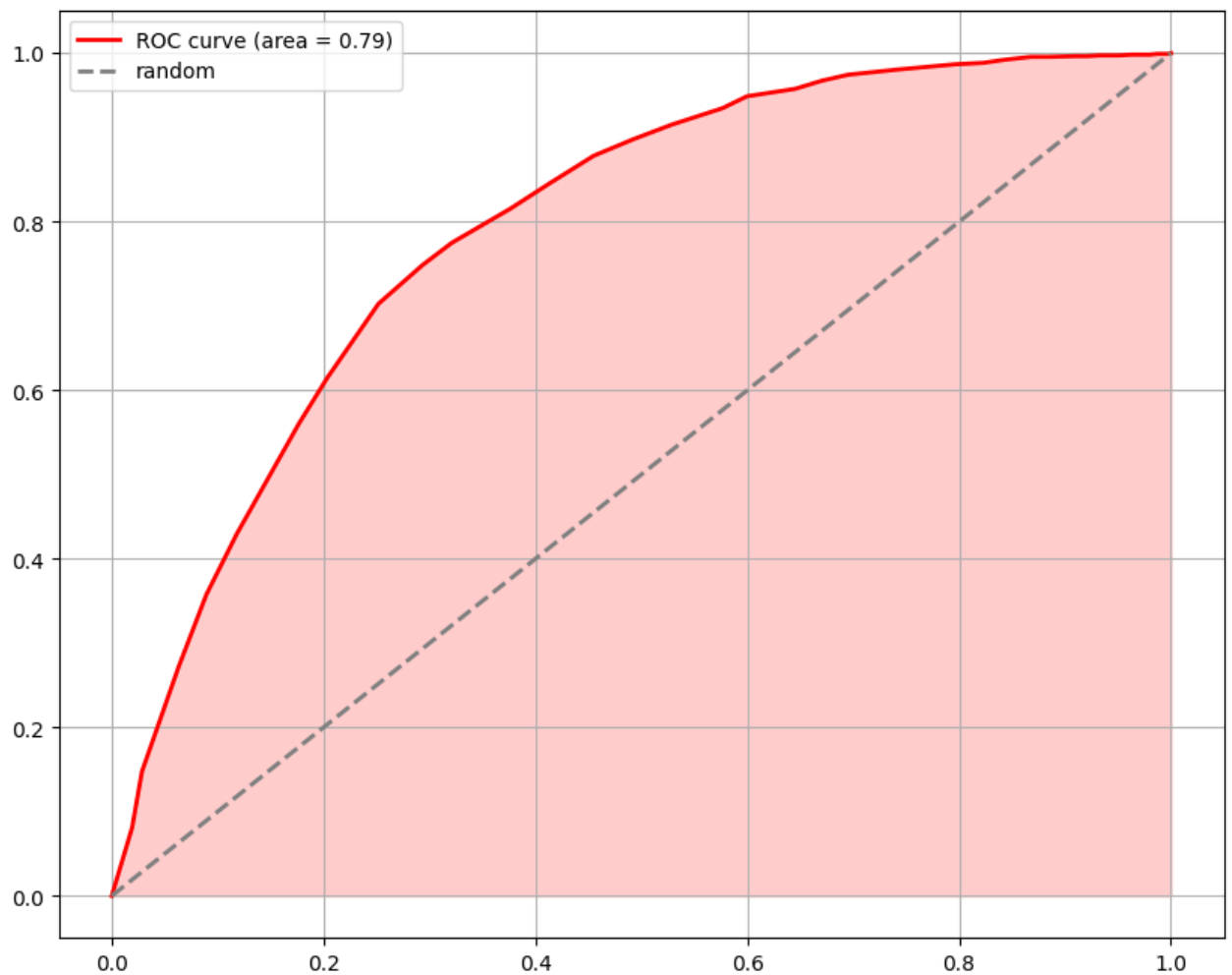
```

Confusion Metrix Q9:
[[882 417]
 [255 878]]
Classification_metrics Q9

```

	precision	recall	f1-score	support
0.0	0.78	0.68	0.72	1299
1.0	0.68	0.77	0.72	1133
accuracy			0.72	2432
macro avg	0.73	0.73	0.72	2432
weighted avg	0.73	0.72	0.72	2432

```
# plot the ROC Curve
plt.figure(figsize=(10, 8))
plt.plot(fpr, tpr, color="red", lw=2, label=f'ROC curve (area = {roc_auc_score(fpr, tpr)})')
plt.fill_between(fpr, tpr, alpha=0.2, color="red")
plt.plot([0, 1], [0, 1], color="gray", lw=2, linestyle='--', label="random")
plt.legend()
plt.grid()
plt.show()
```



✓ Analysis 10

Build a classification model that predicts whether a professor receives a “pepper” from all available factors. Comment on how this model compares to the “average rating only” model. Make sure to include quality metrics such as AU(RO)C and also address class imbalances.

```
#Analysis 10
print()
print()
print("Analysis 10")
print()
X = merged_data_5[["Average Rating", "Male Gender", "Female Gender", '
y = merged_data_5["Received a Pepper"]
class_distribution = y.value_counts(normalize=True)
#check distribution to see is it imbalanced, result no pepper : pepper
print(class_distribution)
```

Analysis 10

Received a Pepper

0.0 0.534046

1.0 0.465954

Name: proportion, dtype: float64

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y
)
# Use SMOTE to resample equal sized sample (balancing)
smote = SMOTE()
X_train_resample, y_train_resample = smote.fit_resample(X_train, y_train)

# Check class distribution after resampling
print("Class Distribution After Resampling:", y_train_resample.value_counts())
```

Class Distribution After Resampling: Received a Pepper

0.0 0.5

1.0 0.5

Name: proportion, dtype: float64

```
# similarly, use logistic model
```

```
logistic_model = LogisticRegression(penalty='l2', C=1.0) # or apply de
```

```

#logistic_model = LogisticRegression()
# or I can use logistic_model = LogisticRegression(class_weight="balanced")
logistic_model.fit(X_train_resample, y_train_resample)

y_pred = logistic_model.predict(X_test)
y_pred_prob = logistic_model.predict_proba(X_test)[:, 1]

roc_auc = roc_auc_score(y_test, y_pred_prob)
fpr, tpr, thresholds = roc_curve(y_test, y_pred_prob)
conf_matrix = confusion_matrix(y_test, y_pred)
classification_metrics = classification_report(y_test, y_pred, output_digits=2)

print({
    "Class Distribution" : class_distribution.to_dict(),
    "ROC AUC" : roc_auc,
    "Confusion Matrix" : conf_matrix.tolist(),
    "Classification Metrics" : classification_report(y_test, y_pred, output_digits=2)
})
print("Confusion Matrix Q10:")
print(conf_matrix)
print("Classification_metrics Q10:")
print(classification_report(y_test, y_pred))

```

```

{'Class Distribution': {0.0: 0.5340460526315789, 1.0: 0.46595394736842105},
 'Confusion Matrix Q10':
 [[844 455]
 [252 881]]
 'Classification_metrics Q10':

```

	precision	recall	f1-score	support
0.0	0.77	0.65	0.70	1299
1.0	0.66	0.78	0.71	1133
accuracy			0.71	2432
macro avg	0.71	0.71	0.71	2432
weighted avg	0.72	0.71	0.71	2432

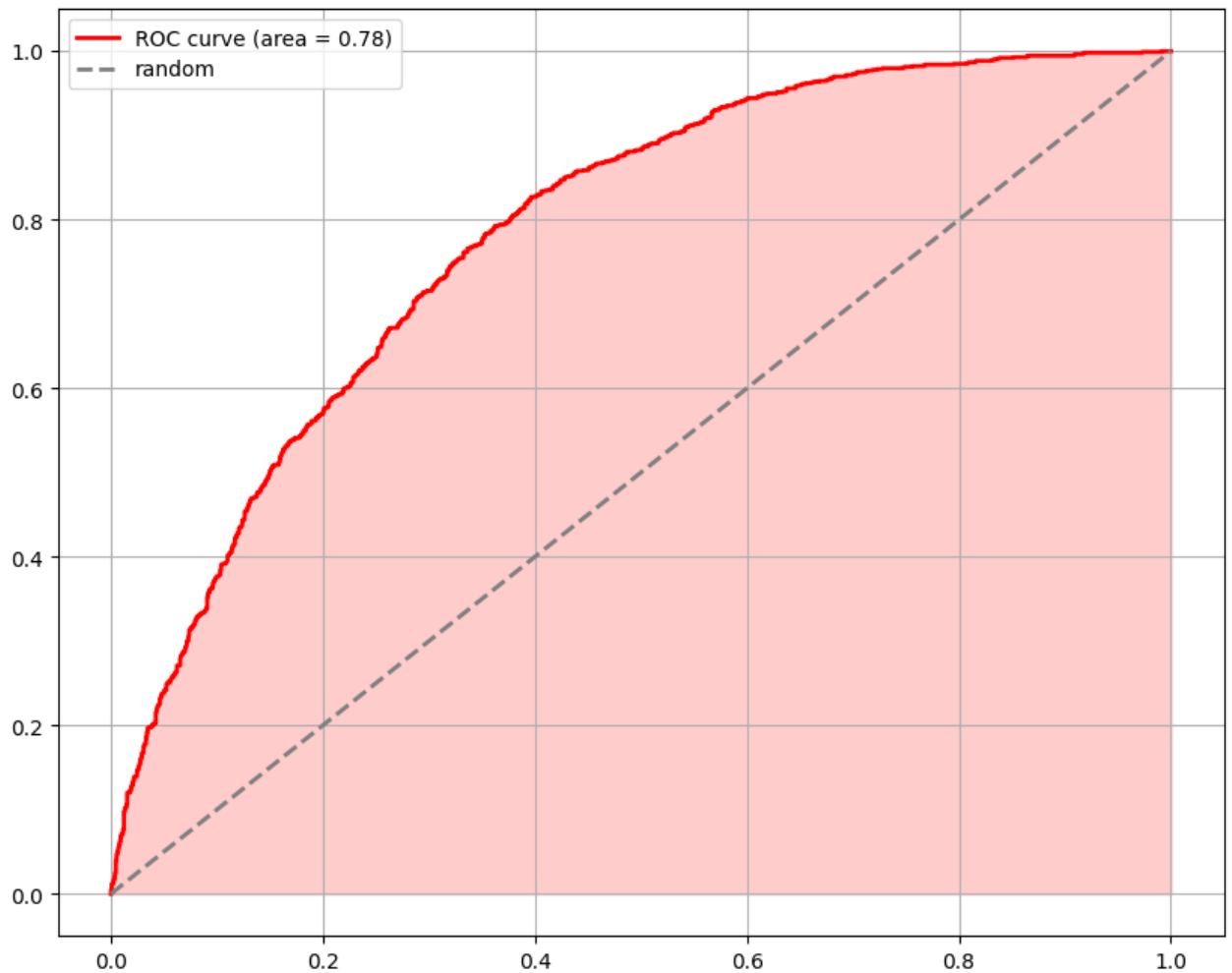
```

/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:114:
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

```

Increase the number of iterations (max_iter) or scale the data as shown in the following link:
<https://scikit-learn.org/stable/modules/preprocessing.html>
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
n_iter_i = _check_optimize_result(

```
import matplotlib.pyplot as plt
plt.figure(figsize=(10, 8))
plt.plot(fpr, tpr, color="red", lw=2, label=f'ROC curve (area = {roc_auc})')
plt.fill_between(fpr, tpr, alpha=0.2, color="red")
plt.plot([0, 1], [0, 1], color="gray", lw=2, linestyle='--', label="random")
plt.legend()
plt.grid()
plt.show()
```



#Analysis for Extra Credit

```

#logistic: Group by State, and see is there a difference in the mean (
statewise_anlaysia = merged_data_cleaned.groupby("US State").agg({
    "Average Rating" : "mean"
})

statewise_anlaysia_sorted = statewise_anlaysia.sort_values(by="Average
print(statewise_anlaysia_sorted)

#Noticed the mean value seems different of each state.
import matplotlib.pyplot as plt

statewise_anlaysia_sorted.reset_index(inplace=True)

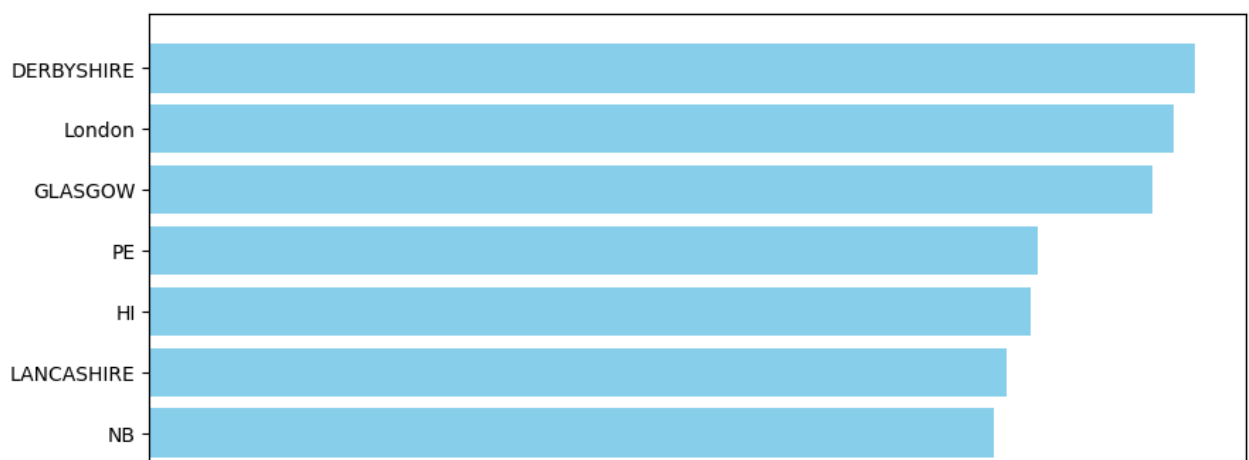
plt.figure(figsize=(10, 6))
plt.barh(statewise_anlaysia_sorted["US State"], statewise_anlaysia_so
plt.gca().invert_yaxis()
plt.show()

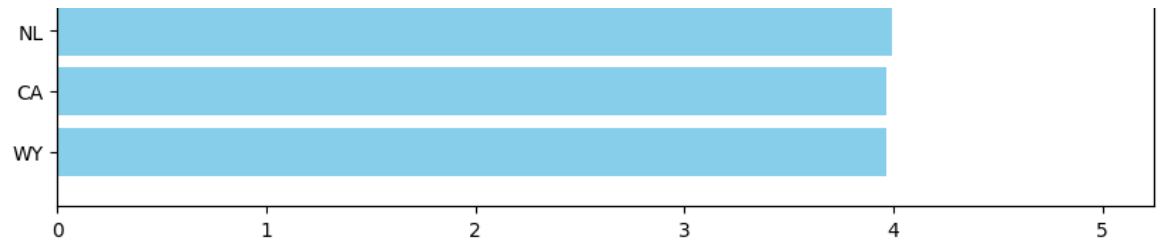
from scipy.stats import kruskal

print(kruskal(*[group["Average Rating"].values for name, group in merg

```

US State	Average Rating
DERBYSHIRE	5.000000
London	4.900000
GLASGOW	4.800000
PE	4.250000
HI	4.218919
LANCASHIRE	4.100000
NB	4.043750
NL	3.992857
CA	3.968323
WY	3.966667





```
KruskalResult(statistic=np.float64(227.2366816550655), pvalue=np.float64(0.00010000000000000001))
```

```
!pip -q install "nbconvert[webpdf]"
```

45.9/45.9 MB 26.7 MB/s eta 0s

```
NB_PATH = "/content/drive/MyDrive/Colab Notebooks/PScapstone.ipynb" ;
out_DIR = "/content/drive/MyDrive/PScapstone/exports" #
out_NAME = "CapstoneProject.pdf" ;

import os
os.makedirs(OUT_DIR, exist_ok=True)
```

```
print("finished: ", f"{OUT_DIR}/{OUT_NAME}")
```

```
[NbConvertApp] Converting notebook /content/drive/MyDrive/Colab Notebook
[NbConvertApp] WARNING | Alternative text is missing on 17 image(s).
[NbConvertApp] Building PDF
Downloading Chromium 140.0.7339.16 (playwright build v1187) from https://cdn.playwright.org/1.49.0/chromium-140.0.7339.16-linux.zip
173.7 MiB [ ] 0% 0.0s173.7 MiB [ ] 0% 25.1s173.7 MiB [ ] 0% 16.4s173.7 MiB [ ] 0% 16.4s
Chromium 140.0.7339.16 (playwright build v1187) downloaded to /root/.cache/ms-playwright/140.0.7339.16
Downloading FFMPEG playwright build v1011 from https://cdn.playwright.org/1.49.0/ffmpeg-1011-linux.zip
2.3 MiB [ ] 0% 0.0s2.3 MiB [ ] 2% 0.7s2.3 MiB [ ] 8% 0.5s2.3 MiB [ ] 19% 0.5s
FFMPEG playwright build v1011 downloaded to /root/.cache/ms-playwright/ffmpeg-1011
Downloading Chromium Headless Shell 140.0.7339.16 (playwright build v1187) from https://cdn.playwright.org/1.49.0/chromium-headless-shell-140.0.7339.16-linux.zip
104.3 MiB [ ] 0% 0.0s104.3 MiB [ ] 0% 18.1s104.3 MiB [ ] 0% 13.0s104.3 MiB [ ] 0% 13.0s
Chromium Headless Shell 140.0.7339.16 (playwright build v1187) downloaded to /root/.cache/ms-playwright/chromium-headless-shell-140.0.7339.16
[NbConvertApp] PDF successfully created
[NbConvertApp] Writing 924185 bytes to /content/drive/MyDrive/PScapstone/exports/CapstoneProject.pdf
finished: /content/drive/MyDrive/PScapstone/exports/CapstoneProject.pdf
```